

CITAS Demonstration Scenarios

Two empirically anchored scenarios were encoded, run against CITAS v2.1, and documented to the ODD protocol. The headline result is a null: in the safety-critical scenario, AI benefit never turns negative across the reliability range — and two isolating sweeps plus a bit-for-bit positive control show this is a real property of the configuration, not of support level, process profile, or the harness.

1 Summary of findings

Every number in this report comes from an executed run. Status labels mark what is established, what was corrected during implementation, and what remains a hypothesis.

VERIFIED **AI support produces a clear, positive effectiveness gain in both scenarios.** Scenario A +1.95 [+1.40, +2.51]; Scenario B +1.06 [+0.63, +1.49] (paired, N = 200, identical seeds). The CIs do not overlap.

VERIFIED **Scenario A shows no zero crossing of AI benefit at any reliability from 0.30 to 0.95,** at trust 0.65 or 0.85. The thesis located a crossing near 48% on its baseline configuration.

VERIFIED **Neither the AI support level nor the process profile explains the missing crossing.** Raising support to 0.80, and separately resetting all four process parameters to defaults, still yields no crossing. A positive control reproduces the thesis crossing bit-for-bit, so the harness is sound.

HYPOTHESIS

The remaining candidates are sprint count (12 vs 8) and dependency density / complexity. Both have a plausible mechanism; neither has been tested. The paper must not attribute the null harm to process discipline.

VERIFIED

The thesis's results_v20/ data is genuine v2.0, not a hybrid. It reproduces bit-for-bit from the pre-v2.1 commit. Only the documentation wording about the config switch needs correcting.

2 Scope and deliverables

Two scenarios anchored in published case studies were encoded as configuration, run against CITAS v2.1, and documented to the ODD protocol. The deliverables:

DELIVERABLE	LOCATION	NOTES
Scenario configurations	scenarios/__init__.py	Two configs as commented dicts following the thesis PRESETS convention; REP/DER/ASS basis code and justification on every parameter; anchor citations verbatim
Runner	run_scenarios.py	Paired arms (A, B), reliability sweep (A), trust × reliability grid (A). Refuses to run unless model is 2.1.0 with anchor trust
Supplementary runner	run_scenarios_support_sweep.py	Isolating sweeps at support 0.80, with a --default-process flag
Results	results_scenarios/	All CSVs plus run_report*.json recording model version, seeds, replication counts, timing
Figures	results_scenarios/figures/	Four figures following make_figures.py conventions; each opened and checked
ODD documentation	scenarios/README.md	Grimm et al. (2020) structure; intended as a paper appendix
Results summary	results_scenarios/RESULTS_SUMMARY.md	Paper-ready numbers, with interpretation and open questions

3 Approach and corrections made during implementation

Anchoring and provenance

Scenario A — safety-critical embedded (electronic brake control unit with a coupled instrument cluster, ISO 26262). Quantitative anchor: Van Schooenderwoert (2006), carried with the caveat that it is an industry experience report by the project's technical lead, not an independent study. Process and regulatory characteristics from Diebold & Mayer (2017) and Heeager & Nielsen (2020). **Scenario B** — large-scale enterprise information system (public-sector pension provider, ~2,500 stories, 12 parallel teams),

anchored in Dingsøyr, Moe & Seim (2018); one representative team modelled. Every parameter carries a basis code — **REP** reported, **DER** derived, **ASS** assumed — in the code itself, so provenance survives outside the paper.

Parameter mapping

All 19 parameter names in the scenario specification are verbatim identifiers of `SimulationConfig`; no renaming was needed. Three parameters the model requires but the specification omitted were added, all coded **ASS**:
`female_proportion` 0.50 (neither anchor reports gender composition),
`team_engagement_baseline` 0.65 (stakes already carried by consequentiality),
`random_seed` 42 (matches the thesis base seed).

Three design decisions

CORRECTED **Four-week-increment variant dropped.** CITAS abstracts over calendar time; per-member capacity per sprint is a fixed constant and iteration length is not a parameter. Halving sprints would answer a different question; a linear capacity multiplier would encode an unevidenced assumption. Recorded in the ODD as outside scope; Heeager & Nielsen's cadence-strain finding flagged as future work needing a documentation-overhead mechanism.

CORRECTED **Skill narrative reframed.** Agent skill is drawn from a fixed distribution with no learning; Scenario A's inexperienced staffing is operationalised through reduced collective memory (0.50) and coordination capability (0.60). `skill_diversity` is a computed output, not an input. Both recorded as stated limitations.

CORRECTED **Baseline arm redefined.** The specification's `ai_support_level = 0` is not a no-AI condition — it still runs the AI allocator, the shared-cognition assistant and the misfit penalty. All baselines use `use_ai=False`, matching the thesis `paired_ai_benefit` convention, so results are comparable to Chapter 6.

Experimental discipline

Replication	200 paired runs for arms and sweeps; 40 per grid cell (matches thesis SENS_REPS)
Seeds	seed = 42 + i, identical within each with/without pair and across every grid cell — common random numbers, so cell-to-cell differences reflect the parameter, not sampling noise
Uncertainty	95% CI = mean $\pm$ 1.96·SE; standard errors reported per grid cell
Version guard	Runner exits unless MODEL_VERSION = "2.1.0" and anchor is trust. The anchor: "actual" switch reverts only the perception anchor — a hybrid — and is not used
Compute	Main run 27,680 simulations in 92.5 s; each supplementary sweep 11,200 in 37 s

4 Integrity checks

VERIFIED **results_v20/ is genuine v2.0 data.** Its

exp1_ci_levels.csv and exp2_trust_scenarios.csv reproduce with zero difference from the code at commit 50c9266, where all four v2.1 mechanics were absent. The overlay in the thesis Fig 5 is correctly labelled. What remains inaccurate is only the weights.yaml comment and Appendix B.6/C.4.3 wording claiming the switch restores v2.0 exactly.

VERIFIED **Positive control.** The scenarios harness, run on the thesis

crossing configuration (defaults, trust 0.85, support 0.80, N = 60), reproduces results/ai_reliability_crossing.csv exactly — benefit -1.26 [-2.23, -0.29] at reliability 0.30, crossing between 0.45 and 0.50.

VERIFIED **Nothing existing was touched.** git diff across results/,

results_v20/, figures/, make_figures.py and every model file is empty. No TODO markers in any deliverable.

5 Results

5.1 Paired arms — with AI vs without AI

METRIC (N = 200)	SCENARIO A	SCENARIO B
Team effectiveness, with / without AI	77.82 / 75.87	75.04 / 73.99
AI benefit (paired Δ effectiveness)	+1.95 [+1.40, +2.51]	+1.06 [+0.63, +1.49]
Share of runs with positive Δ	0.695	0.650
Δ defect rate (pp)	-3.46 [-4.69, -2.23]	-3.50 [-4.34, -2.65]
Δ velocity	+1.12 [+0.62, +1.62]	+0.47 [+0.02, +0.93]
Δ completion rate (pp)	+2.50 [+1.30, +3.70]	+0.34 [-0.54, +1.22]
Δ decision quality	+9.38	+9.25
Δ member sustainability	-0.44	-0.76
Trust calibration, with-AI arm (%)	93.75	93.79

The gain is larger in A and carried by both quality and throughput. In B it is carried almost entirely by quality and decision quality: the completion-rate CI includes zero. Both scenarios show a small negative effect on member sustainability — AI support slightly raises workload pressure even as output improves.

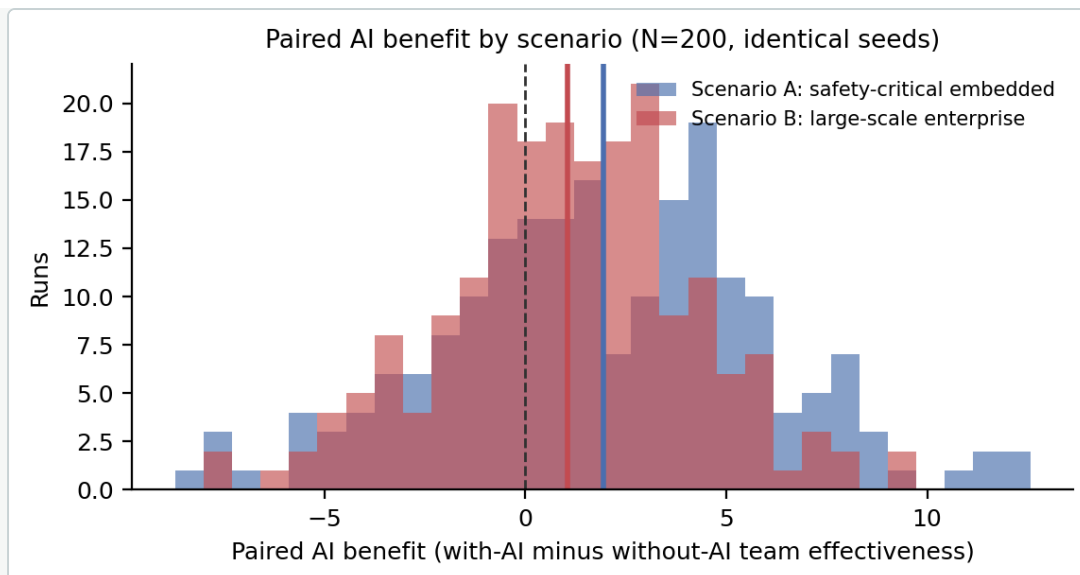


Figure 1. Distribution of paired AI benefit by scenario. Vertical lines mark the scenario means; dashed line marks zero.

5.2 Scenario A reliability sweep — no zero crossing

CONFIGURATION	TRUST	CROSSINGS	BENEFIT AT 0.30		
Scenario A, support 0.70 (main run)	0.65	0	+0.60	[+0.06, +1.15]	+2.10
	0.85	0	+0.08	[-0.52, +0.68]	+2.43
Support raised to 0.80	0.65	0	+0.42	[-0.14, +0.98]	+2.28
	0.85	0	+0.24	[-0.34, +0.82]	+2.61
Support 0.80 + process reset to defaults	0.65	0	+0.27	[-0.33, +0.88]	+2.29
	0.85	0	+0.32	[-0.31, +0.95]	+2.86
Thesis configuration (positive control)	0.85	1	-1.26	[-2.23, -0.29]	+2.36

In every Scenario A variant the benefit stays positive across the whole range (N = 200 per point). Raising support lifts the ceiling by about +0.18 at reliability 0.95 without creating a negative region. Resetting process lowers low-reliability benefit by roughly 0.11–0.14 — a small buffer in the expected direction — but

never flips the sign. The positive-control row shows the same harness recovering the thesis crossing on the thesis configuration.

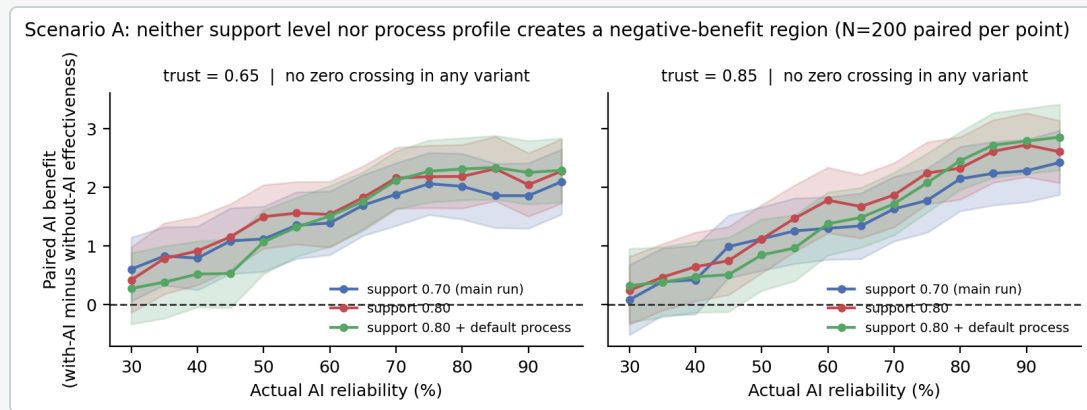


Figure 2. Scenario A benefit versus reliability under three variants: support 0.70, support 0.80, and support 0.80 with default process. Shaded bands are 95% CIs; the dashed line is zero.

What this establishes, and what it does not. Support level and process profile are ruled out as explanations, and the harness is validated. What remains different from the thesis run is Scenario A's team size (5 vs 6), sprint count (12 vs 8), backlog mix, task complexity (0.72 vs 0.58), dependency density (0.45 vs 0.25) and CI baselines. Two carry a plausible mechanism — more sprints give learned trust longer to converge toward true reliability, and higher coordination need enlarges the assistant's upside — but these are untested hypotheses. The paper should present the null harm as *conditional on the scenario configuration*.

5.3 Scenario A trust × reliability grid

STATISTIC (196 CELLS, N = 40 EACH)		VALUE
Per-cell standard error of Δ effectiveness	0.52 – 0.76 (median 0.63)	
Cells with negative mean benefit	16 of 196 (8.2%)	
... among over-trust cells (trust > reliability)		17.6%
... among calibrated / under-trust cells		0.0%
Most negative cell (trust 0.90, reliability 0.35)		-0.86 (SE 0.67)
Most positive cell (trust 0.90, reliability 0.95)		+2.45 (SE 0.60)

The sign structure is clean — every negative cell lies in the over-trust half — but the magnitude of harm is small and no individual cell is statistically distinguishable from zero at $N = 40$. The surface, not any cell, is the result. Calibration forms a diagonal ridge: at trust 0.65 it peaks at 96.0% (reliability 0.70) and falls to 81.4% at 0.95, because perception converges at 0.05 per sprint and a moderately trusting team ends up under-trusting a highly reliable assistant — the under-reliance regime of Lee & See (2004), appearing as intended in v2.1.

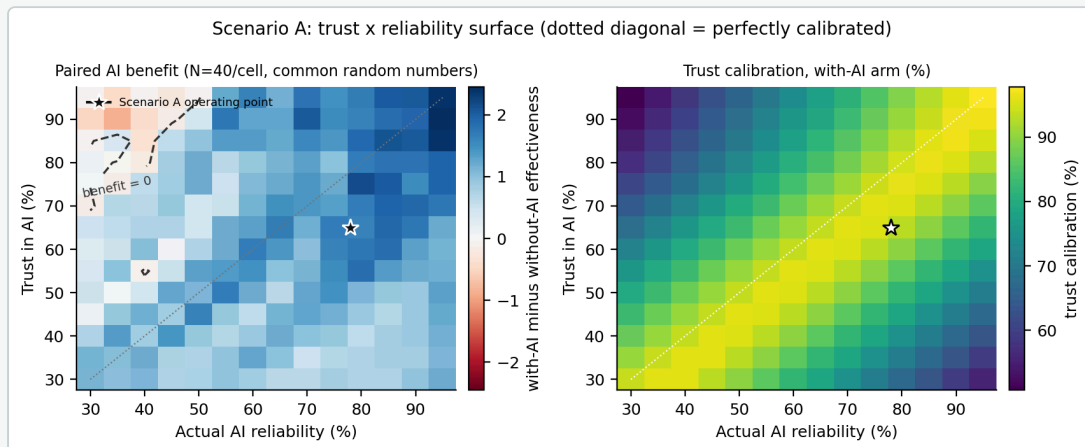


Figure 3. Scenario A surfaces. Left: paired AI benefit, dashed contour at zero. Right: trust calibration. Dotted diagonal marks perfect calibration; the star is Scenario A's operating point (trust 0.65, reliability 0.78).

6 Points for the paper

1. **The no-crossing result needs explicit framing.** It is not a contradiction of the thesis; it is a demonstration that over-trust harm is conditional. Attribute it to configuration, not to process discipline — the isolating test refutes that.
2. **Scenario B's throughput gain is null.** Present its benefit as a quality and decision-quality effect, not a delivery effect.
3. **Sustainability is slightly negative in both scenarios.** Small but consistent in sign; a reviewer will ask.
4. **Calibration falls at high reliability under moderate trust.** Correct behaviour, but counter-intuitive; explain the learning-rate mechanism.
5. **Do not quote individual grid cells.** Quote the surface and its sign structure.

6. **Documentation fix.** Reword the `weights.yaml` comment and Appendix B.6/C.4.3: the anchor switch reverts one of four v2.1 changes, not all.

7 Proposed next steps

- **NOT RUN** Pin the mechanism: the default-process / support-0.80 configuration with `number_of_sprints = 8`, and separately with `dependency_density = 0.25`. About 40 seconds each. If either restores the crossing, the paper can name the mechanism.
 - **NOT RUN** Optionally, the same isolating ladder on Scenario B, to check whether the calibrated-reliance pattern generalises.
 - Draft the "operation" and "application and use" sections from `RESULTS_SUMMARY.md`, with `scenarios/README.md` as the appendix.
-

A Appendix — reproduction

From the repository root, with the project virtual environment active:

COMMAND	PRODUCES
<code>python run_scenarios.py</code>	paired arms, sweep, grid → <code>results_scenarios/</code>
<code>python run_scenarios_support_sweep.py 0.80</code>	supplementary sweep at support 0.80
<code>python run_scenarios_support_sweep.py 0.80 --default-process</code>	isolating sweep, process reset to defaults
<code>python make_scenario_figures.py</code>	four figures → <code>results_scenarios/figures/</code>

Runs are deterministic given the seed rule; re-running reproduces every number above exactly.